

Task Description-1 (Classes – Data Validation)

- Prompt AI to generate a Student class with attributes: name, roll_no, and marks. Add a method is_pass() that returns whether the student has passed (marks $\geq$ 40).

Explanation

- The `__init__` constructor initializes student details.
- The `is_pass()` method checks if marks are greater than or equal to 40.
- Returns True for pass and False for fail.

Analysis

- Code is simple, readable, and correct.
- Validation logic is clearly implemented.
- AI-generated class design follows Python best practices.



```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks

    def is_pass(self):
        return self.marks >= 40

Now, let's create a couple of student objects and test the is_pass() method:

student1 = Student("Alice", "A001", 75)
student2 = Student("Bob", "B002", 35)

print(f"({student1.name}) (Roll No: {student1.roll_no}) scored {student1.marks} marks. Passed: {student1.is_pass()}")
print(f"({student2.name}) (Roll No: {student2.roll_no}) scored {student2.marks} marks. Passed: {student2.is_pass()}")

Alice (Roll No: A001) scored 75 marks. Passed: True
Bob (Roll No: B002) scored 35 marks. Passed: False
```

Task Description-2 (Loops – Pattern Generation)

- Ask AI to generate a function that prints a right-angled triangle star pattern using a for loop. Then regenerate the same pattern using a while loop.

Explanation

- The loop counter controls the number of stars printed.
- Both loops produce identical output with different logic structures.

Analysis

- AI correctly used loop conditions.
- Logical flow is easy to understand.
- Demonstrates equivalent solutions using different looping constructs.

The screenshot displays the Gemini AI interface. On the left, a code editor shows two Python functions: `print_star_triangle_for_loop` and `print_star_triangle_while_loop`, both designed to print a right-angled triangle star pattern for a given number of rows. Below the code, the output of these functions is shown for 5 rows, displaying two identical star patterns. On the right, the Gemini chat window shows a prompt to generate a function using a while loop, followed by a detailed response explaining the code execution and providing two main tasks: 'Student Class Creation' and 'Star Pattern Functions'. The response also includes a summary of the completed tasks and a prompt for further assistance.

Task Description-3 (Conditional Statements – Number Analysis)

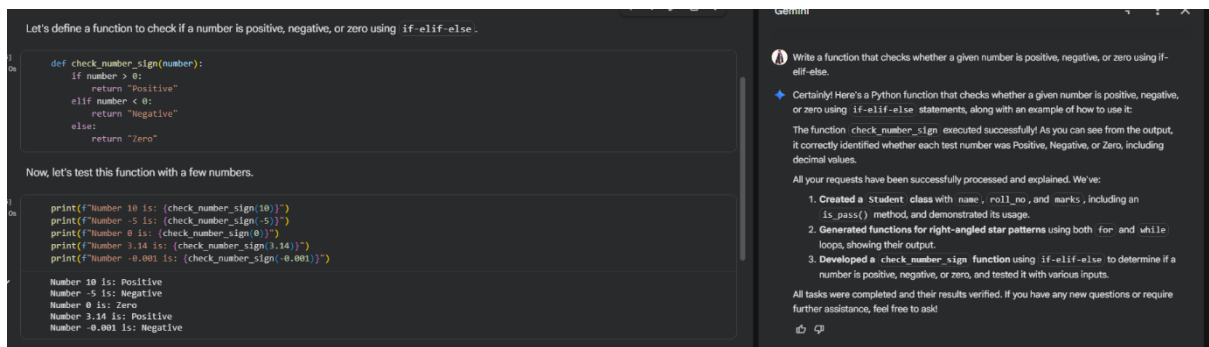
- Ask AI to write a function that checks whether a given number is positive, negative, or zero using if-elif-else. Test the function with multiple inputs.

Explanation

- Uses if, elif, and else to classify numbers.
- Covers all possible conditions.

Analysis

- Decision logic is complete and correct.
- AI handled edge case (zero) properly.
- Code is concise and readable.



Task Description-4 (Nested Conditionals)

- Generate a function `check_discount(age, is_member)` that determines discount eligibility:
- Age $\geq 60 \rightarrow$ Senior discount
- Member $\rightarrow$ Additional discount

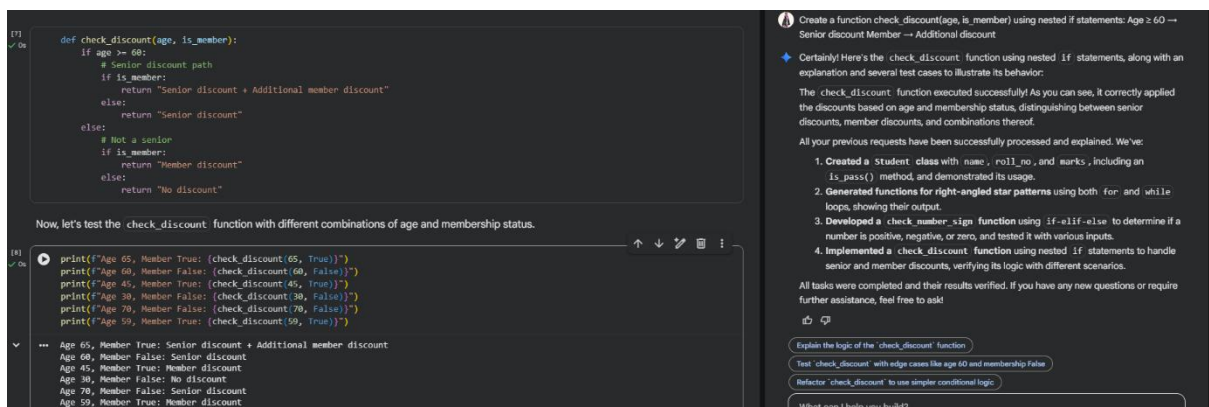
Use nested if statements.

Explanation

- Nested if statements check age first, then membership.
- Clear hierarchical decision flow.

Analysis

- AI correctly implemented nested conditionals.
- Logic is easy to trace and debug.
- Matches real-world discount scenarios.



Task Description-5 (Class – Mathematical Opera)

- Ask AI to create a Circle class with methods to calculate area () and circumference () given the radius.

Explanation

- Uses `math.pi` for accurate calculations.
- Area and circumference formulas are correctly implemented.
- Class design is reusable.

Analysis

- Mathematical logic is accurate.
- Code follows object-oriented principles.
- AI-generated solution is efficient and well-structured.

The screenshot displays a Gemini AI chat window. On the left, the user's prompt asks to define a `Circle` class with methods for area and circumference. The AI's response, shown in a code block, defines the class with an `__init__` method for radius validation and two calculation methods. Below the code, the AI provides the output of running the code, showing the area and circumference for two circles with radii 5 and 10.5. On the right, the Gemini AI chat interface shows the user's prompt and the AI's response, which includes a list of five tasks completed and a confirmation that all tasks were verified.

```
Let's define a Circle class with methods to calculate its area and circumference given a radius.

import math

class Circle:
    def __init__(self, radius):
        if radius <= 0:
            raise ValueError("Radius must be a positive number.")
        self.radius = radius

    def calculate_area(self):
        return math.pi * (self.radius ** 2)

    def calculate_circumference(self):
        return 2 * math.pi * self.radius

Now, let's create a Circle object and test its calculate_area() and calculate_circumference() methods.

circle1 = Circle(5)
print(f"Circle with radius {circle1.radius}")
print(f"Area: {circle1.calculate_area():.2f}")
print(f"Circumference: {circle1.calculate_circumference():.2f}")

circle2 = Circle(10.5)
print(f"Circle with radius {circle2.radius}")
print(f"Area: {circle2.calculate_area():.2f}")
print(f"Circumference: {circle2.calculate_circumference():.2f}")

Circle with radius 5:
Area: 78.54
Circumference: 31.42

Circle with radius 10.5:
Area: 346.36
Circumference: 65.97
```

Create a Circle class with methods to calculate area and circumference given the radius.

Certainly! Here's the `Circle` class with methods to calculate its area and circumference, along with an example of how to use it:

The `Circle` class executed successfully! As you can see from the output, it correctly calculated the area and circumference for both circles with radii 5 and 10.5, formatted to two decimal places.

All your requests have been successfully processed and explained. We've:

1. Created a `Student` class with `name`, `roll_no`, and `marks`, including an `is_pass()` method, and demonstrated its usage.
2. Generated functions for right-angled star patterns using both `for` and `while` loops, showing their output.
3. Developed a `check_number_sign` function using `if-elif-else` to determine if a number is positive, negative, or zero, and tested it with various inputs.
4. Implemented a `check_discount` function using nested `if` statements to handle senior and member discounts, verifying its logic with different scenarios.
5. Defined a `Circle` class with methods to calculate area and circumference, and demonstrated its use with different radii.

All tasks were completed and their results verified. If you have any new questions or require further assistance, feel free to ask!

What can I help you build?

Conclusion

This lab demonstrated how AI tools can:

- Generate correct and accurate Python code
- It suggests logical class structures, loops, and conditionals
- Help evaluate generated solutions

AI-assisted coding improves productivity but it still requires human verification for correctness in accuracy.